

Personal Handbook

1	Bridge	1	11	Sieve	5	21	DSU on Tree(Sack)	10
2	Articulation point	1	12	Fraction finder	5	22	Maximum XOR Subset	10
3	Line container	1	13	FFT v2	5	23	Number of Subset XOR	10
4	Convex Hull	2	14	FFT Mod	6	24	Critical Cities(Dominator)	11
5	KMP	2	15	Heavy light Decomp	6	25	DC DP	13
6	Z function	3	16	Global Min-Cut	7	26	RAND mt19937	15
7	Suffix Array	3	17	Max Clique	7	27	RAND mt19937 Faster	15
8	Manacher	3	18	Dinic 2	8	28	GNU Extensions	15
9	Short ST	3	19	Virtual Tree	8	29	Debug	16
10	Lazy ST	4	20	Kruskal Reconstruction Tree	9			

1 Bridge

```
1 void IS_BRIDGE(int v,int to); // some function to process the found bridge. Printing
  for instance
2 int n; // number of nodes
3 vector<vector<int>> adj; // adjacency list of graph
4
5 vector<bool> visited;
6 vector<int> tin, low;
7 int timer;
8
9 void dfs(int v, int p = -1) {
10     visited[v] = true;
11     tin[v] = low[v] = timer++;
12     bool parent_skipped = false;
13     for (int to : adj[v]) {
14         if (to == p && !parent_skipped) {
15             parent_skipped = true;
16             continue;
17         }
18         if (visited[to]) {
19             low[v] = min(low[v], tin[to]);
20         } else {
21             dfs(to, v);
22             low[v] = min(low[v], low[to]);
23             if (low[to] > tin[v])
24                 IS_BRIDGE(v, to);
25         }
26     }
27 }
28
29 void find_bridges() {
30     timer = 0;
31     visited.assign(n, false);
32     tin.assign(n, -1);
33     low.assign(n, -1);
34     for (int i = 0; i < n; ++i) {
35         if (!visited[i])
36             dfs(i);
37     }
38 }
```

2 Articulation point

```
1 void IS_CUTPOINT(int v); // here v is articulation point, i can print for instance.
2 int n; // number of nodes
```

```
3 vector<vector<int>> adj; // adjacency list of graph
4
5 vector<bool> visited;
6 vector<int> tin, low;
7 int timer;
8
9 void dfs(int v, int p = -1) {
10     visited[v] = true;
11     tin[v] = low[v] = timer++;
12     int children=0;
13     for (int to : adj[v]) {
14         if (to == p) continue;
15         if (visited[to]) {
16             low[v] = min(low[v], tin[to]);
17         } else {
18             dfs(to, v);
19             low[v] = min(low[v], low[to]);
20             if (low[to] >= tin[v] && p!=-1)
21                 IS_CUTPOINT(v);
22             ++children;
23         }
24     }
25     if(p == -1 && children > 1)
26         IS_CUTPOINT(v);
27 }
28
29 void find_cutpoints() {
30     timer = 0;
31     visited.assign(n, false);
32     tin.assign(n, -1);
33     low.assign(n, -1);
34     for (int i = 0; i < n; ++i) {
35         if (!visited[i])
36             dfs(i);
37     }
38 }
```

3 Line container

```
1 struct Line {
2     mutable int k, m, p;
3     bool operator<(const Line& o) const { return k < o.k; }
4     bool operator<(int x) const { return p < x; }
5 };
6
7 // queries for maximum
8 // if we want minimum then make minimum = true
9 bool minimum = true;
10 struct LineContainer : multiset<Line, less<>> {
11     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
```

```

12 static const int inf = LONG_MAX;
13 int div(int a, int b) { // floored division
14     return a / b - ((a ^ b) < 0 && a % b); }
15 bool isect(iterator x, iterator y) {
16     if (y == end()) return x->p = inf, 0;
17     if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
18     else x->p = div(y->m - x->m, x->k - y->k);
19     return x->p >= y->p;
20 }
21 // y = m*x+c; x wiint be the query parameter used later
22 void add(int m, int c) {
23     if( minimum ){
24         m=-m;
25         c=-c;
26     }
27     auto z = insert({m, c, 0}), y = z++, x = y;
28     while (isect(y, z)) z = erase(z);
29     if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
30     while ((y = x) != begin() && (--x)->p >= y->p)
31         isect(x, erase(y));
32 }
33 int get(int x) {
34     assert(!empty());
35     auto l = *lower_bound(x);
36     if( minimum ) return -(l.k * x + l.m);
37     return l.k * x + l.m;
38 }
39 };

```

4 Convex Hull

```

1 struct pt {
2     double x, y;
3     bool operator == (pt const& t) const {
4         return x == t.x && y == t.y;
5     }
6 };
7
8 int orientation(pt a, pt b, pt c) {
9     double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
10    if (v < 0) return -1; // clockwise
11    if (v > 0) return +1; // counter-clockwise
12    return 0;
13 }
14
15 bool cw(pt a, pt b, pt c, bool include_collinear) {
16     int o = orientation(a, b, c);
17     return o < 0 || (include_collinear && o == 0);
18 }
19 bool collinear(pt a, pt b, pt c) { return orientation(a, b, c) == 0; }

```

```

20
21 void convex_hull(vector<pt>& a, bool include_collinear = false) {
22     pt p0 = *min_element(a.begin(), a.end(), [](pt a, pt b) {
23         return make_pair(a.y, a.x) < make_pair(b.y, b.x);
24     });
25     sort(a.begin(), a.end(), [&p0](const pt& a, const pt& b) {
26         int o = orientation(p0, a, b);
27         if (o == 0)
28             return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
29                 < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
30         return o < 0;
31     });
32     if (include_collinear) {
33         int i = (int)a.size()-1;
34         while (i >= 0 && collinear(p0, a[i], a.back())) i--;
35         reverse(a.begin()+i+1, a.end());
36     }
37
38     vector<pt> st;
39     for (int i = 0; i < (int)a.size(); i++) {
40         while (st.size() > 1 && !cw(st[st.size()-2], st.back(), a[i],
41             include_collinear))
42             st.pop_back();
43         st.push_back(a[i]);
44     }
45
46     if (include_collinear == false && st.size() == 2 && st[0] == st[1])
47         st.pop_back();
48
49     a = st;
50 }

```

5 KMP

```

1 vector<int> prefix_function(string s) {
2     int n = (int)s.length();
3     vector<int> pi(n);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i-1];
6         while (j > 0 && s[i] != s[j])
7             j = pi[j-1];
8         if (s[i] == s[j])
9             j++;
10        pi[i] = j;
11    }
12    return pi;
13 }

```

6 Z function

```
1 vector<int> z_function(string s) {
2     int n = s.size();
3     vector<int> z(n);
4     int l = 0, r = 0;
5     for(int i = 1; i < n; i++) {
6         if(i < r) {
7             z[i] = min(r - i, z[i - l]);
8         }
9         while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
10             z[i]++;
11         }
12         if(i + z[i] > r) {
13             l = i;
14             r = i + z[i];
15         }
16     }
17     return z;
18 }
```

7 Suffix Array

```
1 vector<int> sort_cyclic_shifts(string const& s) {
2     int n = s.size();
3     const int alphabet = 256;
4     vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
5     for (int i = 0; i < n; i++)
6         cnt[s[i]]++;
7     for (int i = 1; i < alphabet; i++)
8         cnt[i] += cnt[i-1];
9     for (int i = 0; i < n; i++)
10         p[--cnt[s[i]]] = i;
11     c[p[0]] = 0;
12     int classes = 1;
13     for (int i = 1; i < n; i++) {
14         if (s[p[i]] != s[p[i-1]])
15             classes++;
16         c[p[i]] = classes - 1;
17     }
18     vector<int> pn(n), cn(n);
19     for (int h = 0; (1 << h) < n; ++h) {
20         for (int i = 0; i < n; i++) {
21             pn[i] = p[i] - (1 << h);
22             if (pn[i] < 0)
23                 pn[i] += n;
24         }
25         fill(cnt.begin(), cnt.begin() + classes, 0);
26         for (int i = 0; i < n; i++)
```

```
27         cnt[c[pn[i]]]++;
28         for (int i = 1; i < classes; i++)
29             cnt[i] += cnt[i-1];
30         for (int i = n-1; i >= 0; i--)
31             p[--cnt[c[pn[i]]]] = pn[i];
32         cn[p[0]] = 0;
33         classes = 1;
34         for (int i = 1; i < n; i++) {
35             pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
36             pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << h)) % n]};
37             if (cur != prev)
38                 ++classes;
39             cn[p[i]] = classes - 1;
40         }
41         c.swap(cn);
42     }
43     return p;
44 }
```

8 Manacher

```
1 vector<int> manacher_odd(string s) {
2     int n = s.size();
3     s = "$" + s + "^";
4     vector<int> p(n + 2);
5     int l = 0, r = 1;
6     for(int i = 1; i <= n; i++) {
7         p[i] = min(r - i, p[l + (r - i)]);
8         while(s[i - p[i]] == s[i + p[i]]) {
9             p[i]++;
10        }
11        if(i + p[i] > r) {
12            l = i - p[i], r = i + p[i];
13        }
14    }
15    return vector<int>(begin(p) + 1, end(p) - 1);
16 }
```

9 Short ST

```
1 const int N = 1e5; // limit for array size
2 int n; // array size
3 int t[2 * N];
4
5 void build() { // build the tree (first set positions n through 2n-1 as original
6     // array
7     for (int i = n - 1; i > 0; --i) t[i] = t[i<<1] + t[i<<1|1];
8 }
```

```

8
9 void modify(int p, int value) { // set value at position p
10     for (t[p += n] = value; p > 1; p >>= 1) t[p>>1] = t[p] + t[p^1];
11 }
12
13 int query(int l, int r) { // sum on interval [l, r)
14     int res = 0;
15     for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
16         if (l&1) res += t[l++];
17         if (r&1) res += t[--r];
18     }
19     return res;
20 }

```

10 Lazy ST

```

1 /**
2  * =====
3  * USABILITY GUIDE
4  * =====
5  * 1. Define Combine Operation:
6  * - Sum: combine(a,b) = a + b
7  * - Max: combine(a,b) = max(a,b)
8  * - Min: combine(a,b) = min(a,b)
9  * - Xor: combine(a,b) = a ^ b
10 *
11 * 2. Define Lazy Meaning:
12 * - Range Assignment: lazy[node] = v
13 * - Range Add: lazy[node] += v
14 * - Range Max-Update (chmax): lazy[node] = max(lazy[node], v)
15 *
16 * 3. Modify update_lazy():
17 * Example:
18 * - Range Assignment + Sum:
19 * val[node] = lazy[node];
20 * sum[node] = (r-l+1) * lazy[node];
21 *
22 * - Range Add + Sum:
23 * val[node] += lazy[node];
24 * sum[node] += (r-l+1) * lazy[node];
25 *
26 * - Range Assignment + Max:
27 * val[node] = lazy[node];
28 * sum[node] = lazy[node]; // here sum[] stores max
29 *
30 * 4. Adjust Query Neutral Value:
31 * - Sum: return 0
32 * - Max: return -INF
33 * - Min: return +INF
34 * - Xor: return 0

```

```

35 * - GCD: return 0
36 *
37 * Complexity: O(log N) per operation.
38 * Root is at index 1, responsible for array[0..n-1].
39 */
40 struct segTree {
41     int n;
42     vector<int> val, sum, lazy;
43
44     segTree(int nn) {
45         n = nn;
46         val.resize(n << 2);
47         sum.resize(n << 2);
48         lazy.resize(n << 2);
49     }
50
51     void build(vector<int>& a, int node, int l, int r) {
52         if (l == r) {
53             val[node] = a[l];
54             sum[node] = a[l]; // adjust if using other ops
55         } else {
56             int m = (l + r) >> 1;
57             build(a, node << 1, l, m);
58             build(a, (node << 1) | 1, m + 1, r);
59             val[node] = val[node << 1] + val[(node << 1) | 1]; // combine()
60             sum[node] = sum[node << 1] + sum[(node << 1) | 1]; // combine()
61         }
62     }
63
64     void update_lazy(int node, int l, int r) {
65         if (!lazy[node]) return;
66         int v = lazy[node];
67
68         // Example: Range assignment with sum
69         val[node] = v;
70         sum[node] = (r - l + 1) * v;
71
72         // For other ops: replace this block
73     }
74
75     void push(int node, int l, int r) {
76         if (lazy[node]) {
77             lazy[node << 1] = lazy[(node << 1) | 1] = lazy[node];
78             update_lazy(node, l, r);
79
80             int m = (l + r) >> 1;
81             update_lazy(node << 1, l, m);
82             update_lazy((node << 1) | 1, m + 1, r);
83             lazy[node] = 0;
84         }
85     }
86

```

```

87 void assign_range(int node, int tl, int tr, int l, int r, int v) {
88     if (l > r) return;
89     if (tl == l && tr == r) {
90         lazy[node] = v;
91         if (tl == tr) update_lazy(node, tl, tr);
92         else push(node, tl, tr);
93     } else {
94         push(node, tl, tr);
95         int tm = (tl + tr) >> 1;
96         assign_range(node << 1, tl, tm, l, min(tm, r), v);
97         assign_range((node << 1) | 1, tm + 1, tr, max(tm + 1, l), r, v);
98         sum[node] = sum[node << 1] + sum[(node << 1) | 1]; // combine()
99     }
100 }
101
102 int range_query(int node, int tl, int tr, int l, int r) {
103     if (l > r) return 0; // neutral element for sum
104     if (tl == l && tr == r) {
105         if (tl == tr) update_lazy(node, tl, tr);
106         else push(node, tl, tr);
107         return sum[node];
108     }
109     push(node, tl, tr);
110     int tm = (tl + tr) >> 1;
111     return range_query(node << 1, tl, tm, l, min(r, tm)) +
112         range_query((node << 1) | 1, tm + 1, tr, max(tm + 1, l), r); //
113         combine()
114 }
115
116 int point_query(int node, int tl, int tr, int pos) {
117     if (tl > tr) return 0;
118     if (tl == pos && tr == pos) {
119         update_lazy(node, tl, tr);
120         return val[node];
121     }
122     push(node, tl, tr);
123     int tm = (tl + tr) >> 1;
124     if (pos <= tm)
125         return point_query(node << 1, tl, tm, pos);
126     return point_query((node << 1) | 1, tm + 1, tr, pos);
127 }

```

11 Sieve

```

1 const int MAX_PR = 5'000'000;
2 bitset<MAX_PR> isprime;
3 vi eratosthenesSieve(int lim) {
4     isprime.set(); isprime[0] = isprime[1] = 0;
5     for (int i = 4; i < lim; i += 2) isprime[i] = 0;

```

```

6     for (int i = 3; i*i < lim; i += 2) if (isprime[i])
7         for (int j = i*i; j < lim; j += i*2) isprime[j] = 0;
8     vi pr;
9     rep(i,2,lim) if (isprime[i]) pr.push_back(i);
10    return pr;
11 }

```

12 Fraction finder

```

1 /**
2  * Description: Given  $f$  and  $N$ , finds the smallest fraction  $p/q$  in  $[0, 1]$ 
3  * such that  $f(p/q)$  is true, and  $p, q \leq N$ .
4  * You may want to throw an exception from  $f$  if it finds an exact solution,
5  * in which case  $N$  can be removed.
6  * Usage: fracBS([f](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}
7  * Time:  $O(\log(N))$ 
8  * Status: stress-tested for  $n \leq 300$ 
9  */
10
11 struct Frac { ll p, q; };
12
13 template<class F>
14 Frac fracBS(F f, ll N) {
15     bool dir = 1, A = 1, B = 1;
16     Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
17     if (f(lo)) return lo;
18     assert(f(hi));
19     while (A || B) {
20         ll adv = 0, step = 1; // move hi if dir, else lo
21         for (int si = 0; step; (step *= 2) >= si) {
22             adv += step;
23             Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
24             if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
25                 adv -= step; si = 2;
26             }
27         }
28         hi.p += lo.p * adv;
29         hi.q += lo.q * adv;
30         dir = !dir;
31         swap(lo, hi);
32         A = B; B = !adv;
33     }
34     return dir ? hi : lo;
35 }

```

13 FFT v2

```

1  /**
2  * Description: fft(a) computes  $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot k x / N)$ 
   for all  $k$ .  $N$  must be a power of 2.
3  Useful for convolution:
4   $\text{conv}(a, b) = c$ , where  $c[x] = \sum a[i]b[x-i]$ .
5  For convolution of complex numbers or more than two vectors: FFT, multiply
6  pointwise, divide by  $n$ , reverse(start+1, end), FFT back.
7  Rounding is safe if  $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ 
8  (in practice  $10^{16}$ ; higher for random inputs).
9  Otherwise, use NTT/FFTMod.
10 * Time:  $O(N \log N)$  with  $N = |A| + |B|$  ( $\tilde{1}s$  for  $N=2^{22}$ )
11 * Status: somewhat tested
12 */
13 #pragma once
14
15 typedef complex<double> C;
16 typedef vector<double> vd;
17 void fft(vector<C>& a) {
18     int n = sz(a), L = 31 - __builtin_clz(n);
19     static vector<complex<long double>> R(2, 1);
20     static vector<C> rt(2, 1); // (~ 10% faster if double)
21     for (static int k = 2; k < n; k *= 2) {
22         R.resize(n); rt.resize(n);
23         auto x = polar(1.0L, acos(-1.0L) / k);
24         rep(i, k, 2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
25     }
26     vi rev(n);
27     rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
28     rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
29     for (int k = 1; k < n; k *= 2)
30         for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
31             // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled) /// include-line
32             auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k]; /// exclude-line
33             C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]); /// exclude-line
34             a[i + j + k] = a[i + j] - z;
35             a[i + j] += z;
36         }
37 }
38 vd conv(const vd& a, const vd& b) {
39     if (a.empty() || b.empty()) return {};
40     vd res(sz(a) + sz(b) - 1);
41     int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
42     vector<C> in(n), out(n);
43     copy(all(a), begin(in));
44     rep(i, 0, sz(b)) in[i].imag(b[i]);
45     fft(in);
46     for (C& x : in) x *= x;
47     rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
48     fft(out);
49     rep(i, 0, sz(res)) res[i] = imag(out[i]) / (4 * n);
50     return res;
51 }

```

14 FFT Mod

```

1  /**
2  * Description: Higher precision FFT, can be used for convolutions modulo arbitrary
   integers
3  * as long as  $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$  (in practice  $10^{16}$ 
   or higher).
4  * Inputs must be in  $[0, \text{mod})$ .
5  * Time:  $O(N \log N)$ , where  $N = |A| + |B|$  (twice as slow as NTT or FFT)
6  * Status: stress-tested
7  */
8  #include "FastFourierTransform.h"
9  typedef vector<ll> vl;
10 template<int M> vl convMod(const vl &a, const vl &b) {
11     if (a.empty() || b.empty()) return {};
12     vl res(sz(a) + sz(b) - 1);
13     int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
14     vector<C> L(n), R(n), outs(n), outl(n);
15     rep(i, 0, sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
16     rep(i, 0, sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
17     fft(L), fft(R);
18     rep(i, 0, n) {
19         int j = -i & (n - 1);
20         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
21         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / ii;
22     }
23     fft(outl), fft(outs);
24     rep(i, 0, sz(res)) {
25         ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
26         ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
27         res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
28     }
29     return res;
30 }

```

15 Heavy light Decomp

```

1  include "../data-structures/LazySegmentTree.h"
2
3  template <bool VALS_EDGES> struct HLD {
4      int N, tim = 0;
5      vector<vi> adj;
6      vi par, siz, rt, pos;
7      Node *tree;
8      HLD(vector<vi> adj_)
9          : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
10            rt(N), pos(N), tree(new Node(0, N)) { dfsHld(0); }

```

```

11 void dfsSz(int v) {
12     if (par[v] != -1) adj[v].erase(find(all(adj[v]), par[v]));
13     for (int& u : adj[v]) {
14         par[u] = v;
15         dfsSz(u);
16         siz[v] += siz[u];
17         if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
18     }
19 }
20 void dfsHld(int v) {
21     pos[v] = tim++;
22     for (int u : adj[v]) {
23         rt[u] = (u == adj[v][0] ? rt[v] : u);
24         dfsHld(u);
25     }
26 }
27 template <class B> void process(int u, int v, B op) {
28     for (; rt[u] != rt[v]; v = par[rt[v]]) {
29         if (pos[rt[u]] > pos[rt[v]]) swap(u, v);
30         op(pos[rt[v]], pos[v] + 1);
31     }
32     if (pos[u] > pos[v]) swap(u, v);
33     op(pos[u] + VALS_EDGES, pos[v] + 1);
34 }
35 void modifyPath(int u, int v, int val) {
36     process(u, v, [&](int l, int r) { tree->add(l, r, val); });
37 }
38 int queryPath(int u, int v) { // Modify depending on problem
39     int res = -1e9;
40     process(u, v, [&](int l, int r) {
41         res = max(res, tree->query(l, r));
42     });
43     return res;
44 }
45 int querySubtree(int v) { // modifySubtree is similar
46     return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
47 }
48 };

```

16 Global Min-Cut

```

1 pair<int, vi> globalMinCut(vector<vi> mat) {
2     pair<int, vi> best = {INT_MAX, {}};
3     int n = sz(mat);
4     vector<vi> co(n);
5     rep(i,0,n) co[i] = {i};
6     rep(ph,1,n) {
7         vi w = mat[0];
8         size_t s = 0, t = 0;
9         rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue

```

```

10         w[t] = INT_MIN;
11         s = t, t = max_element(all(w)) - w.begin();
12         rep(i,0,n) w[i] += mat[t][i];
13     }
14     best = min(best, {w[t] - mat[t][t], co[t]});
15     co[s].insert(co[s].end(), all(co[t]));
16     rep(i,0,n) mat[s][i] += mat[t][i];
17     rep(i,0,n) mat[i][s] = mat[s][i];
18     mat[0][t] = INT_MIN;
19 }
20 return best;
21 }

```

17 Max Clique

```

1 /**
2  * Author: chilli, SJTU, Janez Konc
3  * Date: 2019-05-10
4  * License: GPL3+
5  * Source: https://en.wikipedia.org/wiki/MaxCliqueDyn\_maximum\_clique\_algorithm, https://gitlab.com/janezkonc/mcqd/blob/master/mcqd.h
6  * Description: Quickly finds a maximum clique of a graph (given as symmetric bitset
7  * matrix; self-edges not allowed). Can be used to find a maximum independent
8  * set by finding a clique of the complement graph.
9  * Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs
10  * faster for sparse graphs.
11  * Status: stress-tested
12  */
13 typedef vector<bitset<200>> vb;
14 struct Maxclique {
15     double limit=0.025, pk=0;
16     struct Vertex { int i, d=0; };
17     typedef vector<Vertex> vv;
18     vb e;
19     vv V;
20     vector<vi> C;
21     vi qmax, q, S, old;
22     void init(vv& r) {
23         for (auto& v : r) v.d = 0;
24         for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
25         sort(all(r), [](auto a, auto b) { return a.d > b.d; });
26         int mxD = r[0].d;
27         rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
28     }
29     void expand(vv& R, int lev = 1) {
30         S[lev] += S[lev - 1] - old[lev];
31         old[lev] = S[lev - 1];
32         while (sz(R)) {
33             if (sz(q) + R.back().d <= sz(qmax)) return;
34             q.push_back(R.back().i);

```



```

35 vv T;
36 for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
37 if (sz(T)) {
38     if (S[lev]++ / ++pk < limit) init(T);
39     int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
40     C[1].clear(), C[2].clear();
41     for (auto v : T) {
42         int k = 1;
43         auto f = [&](int i) { return e[v.i][i]; };
44         while (any_of(all(C[k]), f)) k++;
45         if (k > mxk) mxk = k, C[mxk + 1].clear();
46         if (k < mnk) T[j++].i = v.i;
47         C[k].push_back(v.i);
48     }
49     if (j > 0) T[j - 1].d = 0;
50     rep(k,mnk,mxk + 1) for (int i : C[k])
51         T[j].i = i, T[j++].d = k;
52     expand(T, lev + 1);
53 } else if (sz(q) > sz(qmax)) qmax = q;
54 q.pop_back(), R.pop_back();
55 }
56 }
57 vi maxClique() { init(V), expand(V); return qmax; }
58 Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
59     rep(i,0,sz(e)) V.push_back({i});
60 }
61 };

```

18 Dinic 2

```

1 // *  $O(\min(E^{1/2}, V^{2/3})E)$  if  $U = 1$ ;  $O(\sqrt{V}E)$  for bipartite matching.
2
3 struct Dinic {
4     struct Edge {
5         int to, rev;
6         ll c, oc;
7         ll flow() { return max(oc - c, 0LL); } // if you need flows
8     };
9     vi lvl, ptr, q;
10    vector<vector<Edge>> adj;
11    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
12    void addEdge(int a, int b, ll c, ll rcap = 0) {
13        adj[a].push_back({b, sz(adj[b]), c, c});
14        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
15    }
16    ll dfs(int v, int t, ll f) {
17        if (v == t || !f) return f;
18        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
19            Edge& e = adj[v][i];
20            if (lvl[e.to] == lvl[v] + 1)

```

```

21         if (ll p = dfs(e.to, t, min(f, e.c))) {
22             e.c -= p, adj[e.to][e.rev].c += p;
23             return p;
24         }
25     }
26     return 0;
27 }
28 ll calc(int s, int t) {
29     ll flow = 0; q[0] = s;
30     rep(L,0,31) do { // 'int L=30' maybe faster for random data
31         lvl = ptr = vi(sz(q));
32         int qi = 0, qe = lvl[s] = 1;
33         while (qi < qe && !lvl[t]) {
34             int v = q[qi++];
35             for (Edge e : adj[v])
36                 if (!lvl[e.to] && e.c >> (30 - L))
37                     q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
38         }
39         while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
40     } while (lvl[t]);
41     return flow;
42 }
43 bool leftOfMinCut(int a) { return lvl[a] != 0; }
44 };

```

19 Virtual Tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 constexpr int MAX_N = 2e5 + 1;
7 constexpr int LG = 18;
8 constexpr int MOD = 998244353;
9
10 int n, a[MAX_N];
11 ll dp[MAX_N], ans;
12 int tin[MAX_N], tout[MAX_N], d[MAX_N], lift[MAX_N][LG], timer;
13 vector<int> adj[MAX_N], vadj[MAX_N], at_a[MAX_N];
14
15 void dfs(int v, int p) {
16     at_a[a[v]].push_back(v);
17     tin[v] = timer++;
18
19     lift[v][0] = p;
20     for (int i = 1; i < LG; i++) { lift[v][i] = lift[lift[v][i - 1]][i - 1]; }
21
22     for (int u : adj[v]) {
23         if (u == p) { continue; }

```

```

24     dfs(u, v);
25 }
26
27 tout[v] = timer++;
28 }
29
30 int is_ancestor(int u, int v) { return tin[u] <= tin[v] && tout[v] <= tout[u]; }
31
32 int lca(int u, int v) {
33     if (is_ancestor(u, v)) { return u; }
34     if (is_ancestor(v, u)) { return v; }
35
36     for (int i = LG - 1; i >= 0; i--) {
37         if (!is_ancestor(lift[u][i], v)) { u = lift[u][i]; }
38     }
39     return lift[u][0];
40 }
41
42 bool sort_tin(const int &a, const int &b) { return tin[a] < tin[b]; }
43
44 vector<int> vtree(const vector<int> &key) {
45     if (key.empty()) return {};
46
47     vector<int> res = key;
48     sort(res.begin(), res.end(), sort_tin);
49
50     for (int i = 1; i < (int)key.size(); i++) {
51         res.push_back(lca(key[i - 1], key[i]));
52     }
53
54     sort(res.begin(), res.end(), sort_tin);
55     res.erase(unique(res.begin(), res.end(), sort_tin), res.end());
56
57     for (int v : res) { vadj[v].clear(); }
58
59     for (int i = 1; i < (int)res.size(); i++) {
60         vadj[lca(res[i - 1], res[i])].push_back(res[i]);
61     }
62
63     return res;
64 }
65
66 int main() {
67     cin >> n;
68
69     for (int i = 0; i < n; i++) { cin >> a[i]; }
70
71     for (int i = 1; i < n; i++) {
72         int u, v;
73         cin >> u >> v;
74         u--;
75         v--;

```

```

76         adj[u].push_back(v);
77         adj[v].push_back(u);
78     }
79
80     dfs(0, 0);
81
82     for (int col = 1; col <= n; col++) {
83         vector<int> vt = vtree(at_a[col]);
84         reverse(vt.begin(), vt.end());
85
86         for (int v : vt) {
87             dp[v] = 1;
88             for (int u : vadj[v]) {
89                 dp[v] *= (dp[u] + 1);
90                 dp[v] %= MOD;
91             }
92             if (a[v] != col) { dp[v]--; }
93
94             ans += dp[v];
95             ans %= MOD;
96             if (a[v] != col) {
97                 for (int u : vadj[v]) {
98                     ans += MOD - dp[u];
99                     ans %= MOD;
100                 }
101             }
102         }
103     }
104
105     cout << ans << '\n';
106 }

```

20 Kruskal Reconstruction Tree

```

1 int getRoot(int u) {
2     if (u == dsu[u]) return u;
3     return dsu[u] = getRoot(dsu[u]);
4 }
5
6 void addEdge(int u, int v) {
7     u = getRoot(u); v = getRoot(v);
8
9     dsu[n] = n;
10    dsu[u] = dsu[v] = n;
11
12    adj[n].push_back(u);
13    if (u != v) adj[n].push_back(v);
14
15    ++n;
16 }

```

```

17
18 void build() {
19     for (int i = 0; i < n; ++i) dsu[i] = i;
20     for (int i = 0; i < m; ++i) addEdge(U[i], V[i]);
21 }

```

21 DSU on Tree(Sack)

```

1 int cnt[maxn];
2 void dfs(int v, int p, bool keep){
3     int mx = -1, bigChild = -1;
4     for(auto u : g[v])
5         if(u != p && sz[u] > mx)
6             mx = sz[u], bigChild = u;
7     for(auto u : g[v])
8         if(u != p && u != bigChild)
9             dfs(u, v, 0); // run a dfs on small childs and clear them from cnt
10    if(bigChild != -1)
11        dfs(bigChild, v, 1); // bigChild marked as big and not cleared from cnt
12    for(auto u : g[v])
13        if(u != p && u != bigChild)
14            for(int p = st[u]; p < ft[u]; p++)
15                cnt[ col[ ver[p] ] ]++;
16    cnt[ col[v] ]++;
17    //now cnt[c] is the number of vertices in subtree of vertex v that has color c.
18    You can answer the queries easily.
19    if(keep == 0)
20        for(int p = st[v]; p < ft[v]; p++)
21            cnt[ col[ ver[p] ] ]--;

```

22 Maximum XOR Subset

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 #define endl '\n'
5
6 int basis[35];
7 void add (int x) {
8     for (int i = 31; i >= 0; --i) {
9         if ((x >> i) & 1) {
10             if (!basis[i]) {
11                 basis[i] = x;
12                 return;
13             }
14             x ^= basis[i];
15         }
16     }

```

```

17 }
18
19 int getMaxXor() {
20     int ans = 0;
21     for (int i = 31; i >= 0; --i) {
22         if ( (ans ^ basis[i]) > ans) {
23             ans ^= basis[i];
24         }
25     }
26     return ans;
27 }
28 void solve () {
29     int n, val;
30     cin >> n;
31     for (int i = 0; i < n; ++i) {
32         cin >> val;
33         add(val);
34     }
35
36     cout << getMaxXor() << endl;
37 }
38
39 int main() {
40     ios::sync_with_stdio(false);
41     cin.tie(nullptr);
42
43     solve();
44
45     return 0;
46 }

```

23 Number of Subset XOR

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 #define endl '\n'
5
6 int basis[35];
7 void add (int x) {
8     for (int i = 31; i >= 0; --i) {
9         if ((x >> i) & 1) {
10             if (!basis[i]) {
11                 basis[i] = x;
12                 return;
13             }
14             x ^= basis[i];
15         }
16     }
17 }

```

```

18
19 int getXorCount() {
20     int ans = 1;
21     for (int i = 31; i >= 0; --i) {
22         if (basis[i] > 0) {
23             ans <<= 1;
24         }
25     }
26     return ans;
27 }
28 void solve () {
29     int n, val;
30     cin >> n;
31     for (int i = 0; i < n; ++i) {
32         cin >> val;
33         add(val);
34     }
35
36     cout << getXorCount() << endl;
37 }
38
39 int main() {
40     ios::sync_with_stdio(false);
41     cin.tie(nullptr);
42
43     solve();
44
45     return 0;
46 }

```

24 Critical Cities(Dominator)

```

1 #include <algorithm>
2 #include <cassert>
3 #include <cmath>
4 #include <stdint>
5 #include <functional>
6 #include <iostream>
7 #include <vector>
8
9 using std::cout;
10 using std::endl;
11 using std::vector;
12
13 // BeginCodeSnip{Tree Class for rdom}
14 template <typename T> class Tree {
15     private:
16         const int root = 0;
17         const vector<int> &parents;
18         const vector<T> &vals;

```

```

19     const int log2dist;
20     vector<vector<int>>> pow2ends;
21     vector<vector<T>>> pow2mins;
22     vector<int> depth;
23
24     public:
25     Tree(const vector<int> &parents, const vector<T> &vals)
26         : parents(parents), vals(vals),
27         log2dist(std::ceil(std::log2(parents.size() + 1))),
28         pow2ends(parents.size(), vector<int>(log2dist + 1)),
29         pow2mins(parents.size(), vector<T>(log2dist + 1)) {
30         assert(parents[root] == -1);
31
32         vector<vector<int>>> children(parents.size());
33         for (int n = 0; n < parents.size(); n++) {
34             if (parents[n] != -1) { children[parents[n]].push_back(n); }
35         }
36         depth = vector<int>(parents.size());
37         vector<int> frontier{root};
38         while (!frontier.empty()) {
39             int curr = frontier.back();
40             frontier.pop_back();
41             for (int n : children[curr]) {
42                 depth[n] = depth[curr] + 1;
43                 frontier.push_back(n);
44             }
45         }
46
47         for (int n = 0; n < parents.size(); n++) {
48             pow2mins[n][0] = vals[n];
49             pow2ends[n][0] = parents[n];
50         }
51         for (int p = 1; p <= log2dist; p++) {
52             for (int n = 0; n < parents.size(); n++) {
53                 int halfway = pow2ends[n][p - 1];
54                 if (halfway == -1) {
55                     pow2ends[n][p] = -1;
56                     pow2mins[n][p] = pow2mins[n][p - 1];
57                 } else {
58                     pow2ends[n][p] = pow2ends[halfway][p - 1];
59                     pow2mins[n][p] =
60                         std::min(pow2mins[n][p - 1], pow2mins[halfway][p - 1]);
61                 }
62             }
63         }
64     }
65
66     /**
67      * @return the min value on the path from the ancestor to its descendant,
68      * not including the ancestor.
69      */
70     T min_val(int ancestor, int desc) {

```

```

71     int dist = depth[desc] - depth[ancestor];
72     T ret = vals[desc];
73     int at = desc;
74     for (int pow = 0; pow <= log2dist; pow++) {
75         if ((dist & (1 << pow)) != 0) {
76             ret = std::min(ret, pow2mins[at][pow]);
77             at = pow2ends[at][pow];
78         }
79     }
80     assert(at == ancestor);
81     return ret;
82 }
83 };
84 // EndCodeSnip
85
86 int main() {
87     int city_num;
88     int flight_num;
89     std::cin >> city_num >> flight_num;
90     vector<vector<int>> adj(city_num);
91     vector<vector<int>> rev_adj(city_num);
92     for (int f = 0; f < flight_num; f++) {
93         int a, b;
94         std::cin >> a >> b;
95         adj[--a].push_back(--b);
96         rev_adj[b].push_back(a);
97     }
98
99     // Variables used for the Euler Tour
100     vector<int> visit_order;
101     // visit_time is initialized with INF so nodes that aren't reachable
102     // during the DFS won't mess up our sdom calculation
103     vector<int> visit_time(city_num, INT32_MAX);
104     vector<int> visit_parent(city_num, -1);
105     vector<bool> visited(city_num, false);
106     std::function<void(int, int)> dfs;
107     dfs = [&](int at, int prev) {
108         if (visited[at]) { return; }
109         visited[at] = true;
110
111         visit_time[at] = visit_order.size();
112         visit_order.push_back(at);
113         visit_parent[at] = prev;
114         for (int next : adj[at]) { dfs(next, at); }
115     };
116     dfs(0, -1);
117
118     // We use a function-based interface instead of a class-based one due to
119     // heavy reliance on previously calculated values
120     vector<int> dsu_parent(city_num, -1);
121     vector<int> dsu_min(city_num, INT32_MAX);
122     std::function<int(int)> find;

```

```

123     find = [&](int x) {
124         if (dsu_parent[x] == -1) { return x; }
125         if (dsu_parent[dsu_parent[x]] != -1) {
126             int parent_res = find(dsu_parent[x]);
127             if (visit_time[parent_res] < visit_time[dsu_min[x]]) {
128                 dsu_min[x] = parent_res;
129             }
130             dsu_parent[x] = dsu_parent[dsu_parent[x]];
131         }
132         assert(dsu_min[x] != INT32_MAX);
133         return dsu_min[x];
134     };
135
136     vector<int> sdom(city_num, -1);
137     for (int i = visit_order.size() - 1; i > 0; i--) {
138         int c = visit_order[i];
139         // Iterate over all nodes with a directed edge to c
140         for (int from : rev_adj[c]) {
141             int find_res = find(from);
142             // Take the node with the earliest visit time in the traversal
143             if (sdom[c] == -1 || visit_time[find_res] < visit_time[sdom[c]]) {
144                 sdom[c] = find_res;
145             }
146         }
147
148         // Link c to its parent in the DFS tree
149         dsu_parent[c] = visit_parent[c];
150         dsu_min[c] = sdom[c];
151     }
152
153     // Initialize the values in the tree
154     vector<std::pair<int, int>> sdom_times(city_num, {INT32_MAX, -1});
155     for (int c : visit_order) {
156         if (c != 0) {
157             // The tree takes the minimum of these pairs, but what we
158             // really want is the node itself- therefore, we store both
159             sdom_times[c] = {visit_time[sdom[c]], c};
160         }
161     }
162     Tree tree(visit_parent, sdom_times);
163
164     vector<int> rdom(city_num, -1);
165     for (int c : visit_order) {
166         if (c != 0) {
167             // Use the definition of the rdom
168             rdom[c] = tree.min_val(sdom[c], c).second;
169         }
170     }
171
172     // Use the properties previously given to compute the idom
173     vector<int> idom(city_num, -1);
174     for (int c : visit_order) {

```

```

175     if (c != 0) { idom[c] = rdom[c] == c ? sdom[c] : idom[rdom[c]]; }
176 }
177
178 // Trace the idom tree from the finish node to the start node,
179 // finding all dominators along the way
180 vector<int> critical{0};
181 int at = city_num - 1;
182 while (at != 0) {
183     critical.push_back(at);
184     at = idom[at];
185 }
186
187 std::sort(critical.begin(), critical.end());
188
189 cout << critical.size() << '\n';
190 for (int i = 0; i < critical.size() - 1; i++) { cout << critical[i] + 1 << ' '; }
191 cout << critical.back() + 1 << endl;
192 }

```

25 DC DP

Divide and Conquer is a dynamic programming optimization.

Preconditions

Some dynamic programming problems have a recurrence of the form:

$$dp(i, j) = \min_{0 \leq k \leq j} \{dp(i-1, k-1) + C(k, j)\}$$

where $C(k, j)$ is a cost function and $dp(i, j) = 0$ when $j < 0$.
Assume:

$$0 \leq i < m, \quad 0 \leq j < n$$

and evaluating C takes $O(1)$ time.

A straightforward evaluation runs in:

$$O(mn^2)$$

There are $m \times n$ states and n transitions per state.

Let $opt(i, j)$ be the value of k that minimizes the recurrence.

If the cost function satisfies the **quadrangle inequality**, then we can prove:

$$opt(i, j) \leq opt(i, j+1)$$

This is called the **monotonicity condition**. For fixed i , the optimal splitting point increases as j increases.

This allows us to reduce the search range when computing transitions.

If we know $opt(i, j)$, then for any $j' < j$:

$$opt(i, j') \leq opt(i, j)$$

So when computing $opt(i, j')$, we only need to search a smaller interval.

Divide and Conquer Optimization

We apply a divide and conquer strategy:

1. Compute $opt(i, \lfloor n/2 \rfloor)$.
2. Recursively compute: - Left half using upper bound $opt(i, n/2)$ - Right half using lower bound $opt(i, n/2)$

By maintaining valid bounds on opt , we reduce the total complexity to:

$$O(mn \log n)$$

Each possible value of $opt(i, j)$ appears in at most $\log n$ recursion levels.

Importantly, the method does not depend on how balanced the splits are. Across each recursion level, every candidate k is used at most twice.

Generic Implementation

Below is a common template. The function `compute` calculates one row i of DP values (`dp_cur`) using the previous row (`dp_before`).

It must be called as:

```
compute(0, n-1, 0, n-1)
```

The function `solve` computes all m rows.

```

1  int m, n;
2  vector<long long> dp_before, dp_cur;
3
4  long long C(int i, int j);
5
6  // compute dp_cur[l] ... dp_cur[r]
7  void compute(int l, int r, int optl, int optr) {
8      if (l > r)
9          return;
10
11     int mid = (l + r) >> 1;
12     pair<long long, int> best = {LLONG_MAX, -1};
13
14     for (int k = optl; k <= min(mid, optr); k++) {
15         best = min(best, {
16             (k ? dp_before[k-1] : 0) + C(k, mid),
17             k
18         });

```

```

19     }
20
21     dp_cur[mid] = best.first;
22     int opt = best.second;
23
24     compute(l, mid - 1, optl, opt);
25     compute(mid + 1, r, opt, opttr);
26 }
27
28 long long solve() {
29     dp_before.assign(n, 0);
30     dp_cur.assign(n, 0);
31
32     for (int i = 0; i < n; i++)
33         dp_before[i] = C(0, i);
34
35     for (int i = 1; i < m; i++) {
36         compute(0, n - 1, 0, n - 1);
37         dp_before = dp_cur;
38     }
39
40     return dp_before[n - 1];
41 }

```

Things to Watch Out For

The hardest part of Divide and Conquer DP is proving the monotonicity of *opt*.

A common sufficient condition is the **quadrangle inequality**:

$$C(a, c) + C(b, d) \leq C(a, d) + C(b, c) \quad \text{for all } a \leq b \leq c \leq d$$

Many Divide and Conquer DP problems can also be solved using the **Convex Hull Trick**, and vice versa. Understanding both techniques is highly recommended.

Möbius Inversion

Möbius Function

The Möbius function $\mu(n)$ is defined as

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1, \\ (-1)^k & \text{if } n \text{ is a product of } k \text{ distinct primes,} \\ 0 & \text{if } n \text{ has a squared prime factor} \end{cases}$$

Key identity:

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & \text{if } n = 1, \\ 0 & \text{if } n > 1. \end{cases}$$

Inversion Formula

If

$$F(n) = \sum_{d|n} f(d),$$

then

$$f(n) = \sum_{d|n} \mu(d) F\left(\frac{n}{d}\right).$$

Equivalently, in Dirichlet convolution form:

$$F = f * 1 \iff f = F * \mu.$$

Common CP Applications

Counting pairs with $\gcd(i, j) = 1$:

$$\sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

Used to:

- Remove overcounting over divisors
- Transform sums over gcd
- Invert divisor summations

Linear Sieve for μ ($O(n)$)

```

1  const int N = 1e6;
2  int mu[N], primes[N], pcnt;
3  bool is_comp[N];
4
5  void mobius_sieve(int n) {
6      mu[1] = 1;
7      for (int i = 2; i <= n; i++) {
8          if (!is_comp[i]) {
9              primes[pcnt++] = i;
10             mu[i] = -1;
11         }
12         for (int j = 0; j < pcnt && i * primes[j] <= n; j++) {
13             int x = i * primes[j];
14             is_comp[x] = true;
15             if (i % primes[j] == 0) {
16                 mu[x] = 0;
17                 break;
18             } else {

```

```

19     mu[x] = -mu[i];
20 }
21 }
22 }
23 }

```

26 RAND mt19937

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
5
6 int main() {
7     int x = rng(); // random 32-bit integer
8     int y = rng() % 100; // 0 to 99
9     cout << x << " " << y << "\n";
10 }

```

27 RAND mt19937 Faster

```

1 mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());
2 unsigned long long x = rng64();

```

28 GNU Extensions

Ordered Set / Ordered Map (PBDS)

Header:

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;

```

Ordered Set

```

1 template<typename T>
2 using ordered_set = tree<
3     T,
4     null_type,
5     less<T>,
6     rb_tree_tag,
7     tree_order_statistics_node_update>;

```

Ordered Map

```

1 template<typename K, typename V>
2 using ordered_map = tree<
3     K,
4     V,
5     less<K>,
6     rb_tree_tag,
7     tree_order_statistics_node_update>;

```

Supported operations

```

1 ordered_set<int> s;
2
3 s.insert(x);
4 s.erase(x);
5
6 // number of elements strictly smaller than x
7 int k = s.order_of_key(x);
8
9 // iterator to k-th (0-indexed) element
10 auto it = s.find_by_order(k);

```

Time complexity: $O(\log n)$

Ordered Multiset

PBDS does not support duplicates directly. Trick: store pairs.

```

1 using pii = pair<int,int>;
2
3 ordered_set<pii> ms;
4 int id = 0;
5
6 ms.insert({value, id++});
7
8 // erase one occurrence
9 ms.erase(ms.lower_bound({value, 0}));

```

Alternative trick (less clean):

```

1 tree<int, null_type, less_equal<int>,
2     rb_tree_tag,
3     tree_order_statistics_node_update> ms;

```

Rope

Header:

```

1 #include <ext/rope>
2 using namespace __gnu_cxx;

```

Rope is useful for fast string modifications.


```

1 rope<char> r;
2
3 r.insert(pos, "abc");
4 r.erase(pos, len);
5 r.replace(pos, len, "xyz");
6
7 char c = r[i];

```

Time complexity:

insert/erase/substring = $O(\log n)$

Common use cases:

- Editor simulation
- Large string modifications
- Persistent-like behavior

29 Debug

Pre-submit:

Write a few simple test cases if sample is not enough.
 Are time limits close? If so, generate max cases.
 Is the memory usage fine?
 Could anything overflow?
 Make sure to submit the right file.

Wrong answer:

Print your solution! Print debug output, as well.
 Are you clearing all data structures between test cases?
 Can your algorithm handle the whole range of input?
 Read the full problem statement again.
 Do you handle all corner cases correctly?
 Have you understood the problem correctly?
 Any uninitialized variables?
 Any overflows?
 Confusing N and M, i and j, etc.?
 Are you sure your algorithm works?
 What special cases have you not thought of?
 Are you sure the STL functions you use work as you think?
 Add some assertions, maybe resubmit.
 Create some testcases to run your algorithm on.

Go through the algorithm for a simple case.

Go through this list again.

Explain your algorithm to a teammate.

Ask the teammate to look at your code.

Go for a small walk, e.g. to the toilet.

Is your output format correct? (including whitespace)

Rewrite your solution from the start or let a teammate do it.

Runtime error:

Have you tested all corner cases locally?

Any uninitialized variables?

Are you reading or writing outside the range of any vector?

Any assertions that might fail?

Any possible division by 0? (mod 0 for example)

Any possible infinite recursion?

Invalidated pointers or iterators?

Are you using too much memory?

Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:

Do you have any possible infinite loops?

What is the complexity of your algorithm?

Are you copying a lot of unnecessary data? (References)

How big is the input and output? (consider scanf)

Avoid vector, map. (use arrays/unordered_map)

What do your teammates think about your algorithm?

Memory limit exceeded:

What is the max amount of memory your algorithm should need?

Are you clearing all data structures between test cases?